

# **TRABAJO PRACTICO INTEGRADOR**

## **VacaBot: Automatizacion del proceso de solicitud de vacaciones**

Alumno: Franco Aglieri

Comision: TUPaD

Fecha: Junio 2026

### **1. RESUMEN EJECUTIVO**

El presente trabajo plantea, desde la perspectiva de un consultor tecnologico, la automatizacion de un proceso administrativo critico en una organizacion ficticia: la gestion de solicitudes de vacaciones del personal. Se analiza el flujo manual actual (AS-IS), se propone una solucion automatizada bajo notacion BPMN 2.0 (TO-BE) y se desarrolla un chatbot simulado en Python que ejecuta dicha logica de punta a punta, validando la coherencia entre el modelo de procesos y la implementacion en codigo.

### **2. IDENTIFICACION DEL PROCESO**

#### **Organizacion ficticia: Crombie S.A.**

PYME del sector software con 30 empleados distribuidos en tres areas (Desarrollo, Soporte y Finanzas). Historicamente la gestion de vacaciones se realizaba en papel y luego se transcribia a una planilla de Excel administrada por RRHH, generando demoras de 3 a 5 dias habiles, errores de carga, perdida de formularios y reclamos por saldos mal calculados.

#### **Proceso seleccionado: Solicitud de vacaciones**

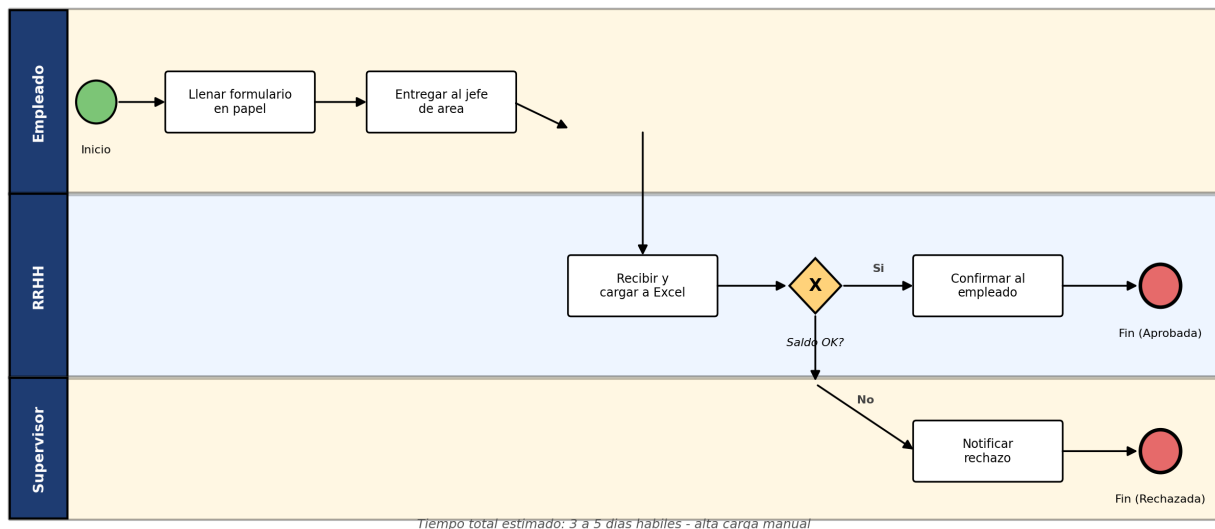
Se eligio este proceso por contar con reglas de negocio claras (saldo de dias por antiguedad, feriados, solapamientos) y por incluir al menos dos puntos de decision logicos, requisito indispensable de la consigna.

### 3. ANALISIS DEL FLUJO ACTUAL (AS-IS)

El flujo manual involucra tres actores: el empleado, el area de RRHH y el supervisor directo. El empleado completa un formulario fisico, lo entrega al jefe de area y RRHH valida el saldo consultando una planilla. Si hay saldo, se confirma; si no, se deriva al supervisor para notificar el rechazo.

- Tareas predominantemente manuales y dependientes de la disponibilidad de RRHH.
- Tiempo total estimado: 3 a 5 dias habiles.
- Sin trazabilidad: no queda registro centralizado del historial de solicitudes por empleado.
- Alta probabilidad de error humano al calcular saldos.

**Diagrama BPMN 2.0 - AS-IS: Solicitud de vacaciones (proceso manual)**



*Figura 1. Diagrama BPMN 2.0 AS-IS del proceso actual.*

### 4. PROPUESTA DE MEJORA (TO-BE)

La propuesta consiste en reemplazar el formulario y la intermediación manual de RRHH por un chatbot administrativo. El bot expone un menú conversacional, valida en tiempo real los datos del empleado contra la base de datos, calcula los días hábiles efectivos (excluyendo fines de semana y feriados) y aplica las reglas de aprobación definidas por la empresa. RRHH queda fuera del camino crítico y solo interviene en casos excepcionales (auditoría).

## 5. MODELO BPMN 2.0 PROPUESTO (TO-BE)

El diagrama TO-BE refleja fielmente la lógica del chatbot e incluye los tres elementos exigidos por la consigna: carriles (lanes) que diferencian Usuario, Sistema/Bot y Supervisor; eventos de inicio y fin para cada camino; y compuertas (gateways) exclusivas para la toma de decisiones.

Diagrama BPMN 2.0 - TO-BE: Solicitud de vacaciones automatizada (Chatbot)

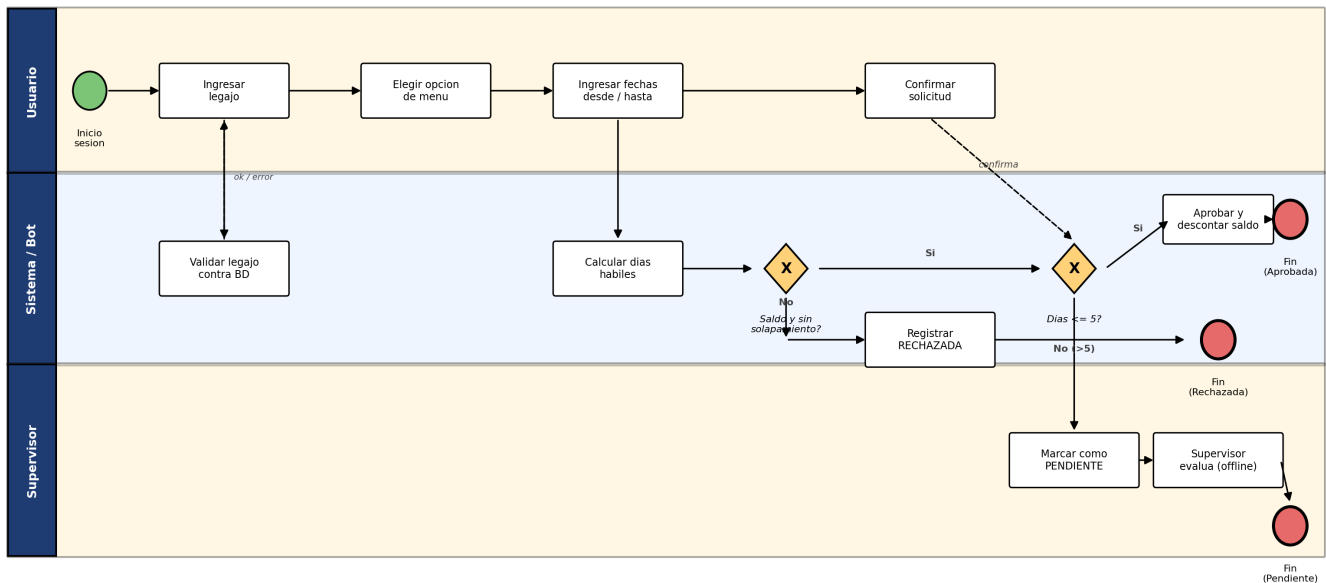


Figura 2. Diagrama BPMN 2.0 TO-BE: flujo automatizado del chatbot.

### Compuertas (gateways) del modelo

- Gateway 1 - Saldo y solapamiento: si el empleado no tiene saldo suficiente o ya posee otra solicitud aprobada/pendiente en el mismo rango, la solicitud se registra automáticamente como RECHAZADA.
- Gateway 2 - Limite de dias: si la solicitud abarca 5 o menos días hábiles, el bot la aprueba automáticamente y descuenta el saldo; si supera ese limite, queda en estado PENDIENTE para validación del supervisor.

## 6. ARQUITECTURA Y STACK TECNOLÓGICO

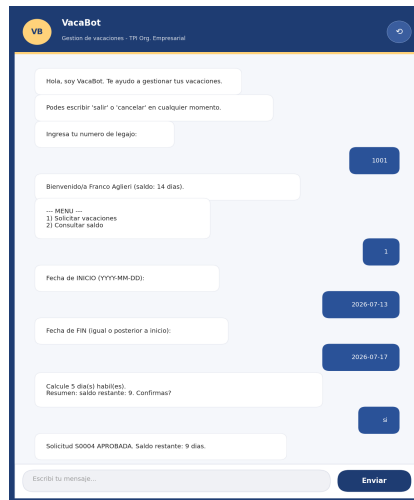


Figura 3. Vista del chatbot web servido en <http://127.0.0.1:5050/>.

### Plataforma y lenguaje

Se optó por una aplicación web liviana escrita en Python 3 sobre el micro-framework Flask 3. La interfaz se sirve como una página HTML estilo chat (similar a WhatsApp Business o una vista de intranet), con un front-end en HTML + CSS + JavaScript que consume una API JSON expuesta por el servidor. La lógica de negocio está encapsulada en el módulo `bot_core.py` de forma turn-based (cada llamada recibe un mensaje y devuelve respuestas), lo que permite reutilizarla también desde una versión CLI sin duplicar código. Se eligió Python por su sintaxis clara, el módulo csv nativo y la facilidad de despliegue de Flask. Alternativas evaluadas: Telegram Bot API (requiere registro de bot y token), WhatsApp Business (API paga) y Node.js con Express (descartado para mantener un único lenguaje).

### Persistencia

Se utilizan archivos CSV como plantilla simulada de base de datos, según lo permite la consigna. Tres tablas: empleados, solicitudes y feriados. El servidor lee, valida y escribe sobre estos archivos en cada interacción, garantizando que el estado del negocio quede persistido entre sesiones. El estado conversacional de cada usuario se mantiene en memoria del servidor, indexado por un identificador único guardado en la cookie firmada de Flask (no se transmite información sensible).

### Herramientas de IA utilizadas

No se integra ningún servicio de IA en tiempo de ejecución: la consigna admite simulación. Si se usaron asistentes de IA en la etapa de diseño para estructurar el flujo BPMN, revisar la lógica del código y resolver el manejo de estado del lado del servidor. Los prompts y respuestas se muestran en la Sección 11.

## 7. DICCIONARIO DE DATOS

### Entidad: empleados

| Campo            | Tipo   | Descripción                                        |
|------------------|--------|----------------------------------------------------|
| legajo           | entero | Identificador único del empleado (clave primaria). |
| nombre           | texto  | Nombre del empleado.                               |
| apellido         | texto  | Apellido del empleado.                             |
| area             | texto  | Área funcional (Desarrollo, Soporte, etc.).        |
| dias_disponibles | entero | Saldo actual de días hábiles disponibles.          |
| antiguedad_anios | entero | Antigüedad en la empresa (informativo).            |

| Campo      | Tipo   | Descripcion                    |
|------------|--------|--------------------------------|
| supervisor | entero | Legajo del supervisor directo. |

**Entidad: solicitudes**

| Campo           | Tipo   | Descripcion                                 |
|-----------------|--------|---------------------------------------------|
| id_solicitud    | texto  | Identificador autogenerado (formato Snnnn). |
| legajo          | entero | Legajo del empleado solicitante.            |
| fecha_solicitud | fecha  | Fecha en la que se carga la solicitud.      |
| fecha_desde     | fecha  | Inicio del periodo solicitado.              |
| fecha_hasta     | fecha  | Fin del periodo solicitado.                 |
| cantidad_dias   | entero | Dias habiles calculados por el bot.         |
| estado          | texto  | APROBADA / RECHAZADA / PENDIENTE.           |
| motivo_rechazo  | texto  | Detalle si el estado es RECHAZADA.          |

## 8. GESTION DE ESTADOS DEL BOT

El chatbot mantiene una clase Sesion que representa la maquina de estados de cada usuario. La memoria del bot garantiza que, luego de cualquier interaccion, el sistema sepa en que paso del proceso se encuentra el usuario y como debe interpretar la siguiente entrada.

| Estado              | Significado                          | Transicion esperada                   |
|---------------------|--------------------------------------|---------------------------------------|
| INICIO              | Recien inicia sesion.                | -> AUTENTICADO al validar legajo.     |
| AUTENTICADO         | Muestra el menu principal.           | -> SOLICITANDO_DESDE / lectura / FIN. |
| SOLICITANDO_DESDE   | Espera fecha de inicio.              | -> SOLICITANDO_HASTA o AUTENTICADO.   |
| SOLICITANDO_HASTA   | Espera fecha de fin.                 | -> VALIDANDO.                         |
| VALIDANDO           | Sistema evalua saldo y solapamiento. | -> CONFIRMANDO o RECHAZADA.           |
| CONFIRMANDO         | Espera si/no del usuario.            | -> APROBADA o DERIVADO_SUPERVISOR.    |
| DERIVADO_SUPERVISOR | Supera el limite automatico.         | -> AUTENTICADO con estado PENDIENTE.  |
| FIN                 | Sesion terminada.                    | Cierra el bot.                        |

## 9. MANUAL DE USUARIO

### Inicio

Desde la carpeta proyecto ejecutar 'pip install flask' la primera vez y luego 'python3 app.py'. La consola informara que el servidor esta escuchando en <http://127.0.0.1:5050/>. Abrir esa direccion en cualquier navegador moderno: aparecera la vista del chat y el bot saludara solicitando el numero de legajo.

### Comandos principales

- 1) Solicitar vacaciones: el bot pide fecha desde y fecha hasta, valida saldo y solapamientos, y resuelve segun los gateways.
- 2) Consultar saldo de dias: muestra el saldo disponible actualizado en tiempo real consultando el CSV de empleados.
- 3) Ver mis solicitudes: lista historica del empleado con estado y motivos de rechazo.
- 4) Salir: cierra la sesion.

### Comandos especiales

- 'salir': finaliza la sesion en cualquier momento del flujo.
- 'cancelar': aborta el sub-flujo actual y vuelve al menu principal sin guardar nada.
- Boton circular de la cabecera (icono de reinicio): limpia la sesion del navegador y reinicia la conversacion como un nuevo usuario.

## 10. PRUEBAS DE ROBUSTEZ (CAMINO INFELIZ)

La excelencia del proyecto se demuestra en la capacidad de manejar errores de entrada y flujos de excepcion. A continuacion se documentan los escenarios negativos probados sobre el simulador junto con la respuesta esperada del bot.

| Escenario             | Entrada                 | Respuesta del bot                                                |
|-----------------------|-------------------------|------------------------------------------------------------------|
| Legajo inexistente    | 9999                    | Mensaje de error y hasta 3 reintentos antes de cerrar la sesion. |
| Legajo con texto      | abc                     | Rechaza por no ser numerico, reintenta.                          |
| Fecha invalida        | 32/15/2026              | Mensaje 'Fecha invalida' y reintento.                            |
| Fecha pasada          | 2020-01-01              | Rechaza por ser anterior a hoy y reintenta.                      |
| Fecha hasta < desde   | Hasta < Desde           | Rechaza y vuelve a pedir la fecha hasta.                         |
| Sin saldo             | Pide 5 dias, saldo = 0  | Registra solicitud como RECHAZADA con motivo.                    |
| Solapamiento          | Mismo rango ya aprobado | RECHAZADA con motivo 'Solapamiento con otra solicitud'.          |
| Solicitud > 5 dias    | 10 dias habiles         | Estado PENDIENTE, derivada al supervisor.                        |
| Confirmacion negativa | Confirmas? -> 'no'      | Cancela la solicitud sin persistir nada.                         |
| Interrupcion Ctrl+C   | Senal SIGINT            | Captura la excepcion y cierra ordenadamente.                     |
| Opcion fuera de menu  | Elegir '9'              | Mensaje 'Opcion no valida' y vuelve a mostrar el menu.           |

### Politica de reintentos

Para toda entrada que pueda fallar (legajo, fechas, opciones numericas) el bot otorga hasta 3 intentos. Si se agotan, vuelve al menu principal o cierra la sesion segun el contexto. El usuario puede escribir 'cancelar' en cualquier momento para abortar el sub-flujo activo y 'salir' para terminar.

## 11. USO DE IA EN LA ETAPA DE DISEÑO

Si bien el bot no integra IA en tiempo de ejecución, durante la etapa de diseño se consultaron asistentes para validar la coherencia del modelo BPMN con la implementación y para revisar el manejo de errores. A continuación se transcriben los prompts más relevantes empleados.

### **Prompt 1 - Diseño del BPMN**

'Tengo que automatizar el proceso de solicitud de vacaciones para una PYME. Que actores y compuertas debería incluir en un diagrama BPMN 2.0 de manera que el flujo sea auditable y permita derivar a un supervisor cuando se supera un umbral?'

La respuesta del asistente sugirió incluir tres carriles (Empleado, Sistema, Supervisor), dos compuertas exclusivas (saldo y umbral de días), y diferenciar los eventos finales por cada camino lógico para no perder trazabilidad. Esta recomendación fue adoptada y se refleja en la Figura 2.

### **Prompt 2 - Camino infeliz**

'Que casos de error debería contemplar un chatbot que pide fechas y un legajo numérico? Quiero garantizar que ninguna entrada del usuario haga caer el sistema.'

El asistente listó: validación de formato de fecha, fecha anterior a hoy, fecha hasta menor a desde, legajo no numérico, interrupción por teclado y opciones fuera del menú. Cada caso fue incorporado a las pruebas de estrés documentadas en la Sección 10.

### **Prompt 3 - Máquina de estados sobre HTTP**

'Como mantengo el estado conversacional de un chatbot que corre en Flask, sabiendo que HTTP es sin estado y cada request es independiente? Necesito que el bot recuerde en qué paso del proceso está cada usuario.'

La sugerencia fue separar la lógica del bot en una función pura turn-based que recibe (estado\_actual, mensaje) y devuelve (nuevo\_estado, respuestas), y guardar ese estado en un diccionario en memoria del servidor indexado por un id único que viaja en la cookie firmada de Flask. Así el cliente solo transporta una referencia y la información sensible nunca sale del servidor. Esa arquitectura se implementó en bot\_core.py (lógica pura) y app.py (transporte HTTP).

## 12. CONCLUSIONES

El trabajo permitió integrar conocimientos de Organización Empresarial (modelado BPMN), Programación (Python, manejo de estado, persistencia) y Análisis de Procesos (identificación de ineficiencias y diseño de soluciones). La coherencia entre el diagrama TO-BE y el código se mantiene en cada gateway: cada compuerta del modelo corresponde a una estructura if/else explícita en la implementación, lo que facilita futuras modificaciones y auditorías.

El siguiente paso natural sería portar el simulador a una plataforma real (Telegram o WhatsApp Business), reemplazar los CSV por una base de datos relacional y agregar notificaciones automáticas al supervisor cuando una solicitud queda en estado PENDIENTE.